

Specificații de modelare și proiectare software E2

Informatică Industrială
– Proiect –
Echipa nr. 47



Cuprins

1. Diagrama UML a claselor.....	3
2. Descrierea arhitecturii aplicației.....	4
a. structura pe module sau straturi (UI, logică, acces la date).....	4
a.1. Stratul de Prezentare (UI).....	4
a.2. Stratul API.....	5
a.3. Stratul de Logică.....	5
a.4. Stratul de Acces la Date (DAL).....	5
b. specificarea arhitecturii sau arhitecturilor folosite (MVVM, MVC, Client-Server, 3-tier, Clean Atchitecture etc.).....	5
3. Designul interfeței grafice.....	6
a. schițe sau mockup-uri ale ferestrelor aplicației.....	6
4. Descrierea claselor principale.....	7
a. rolul fiecărei clase + responsabilități.....	7
5. Planul de testare.....	9
a. definirea cazurilor de testare.....	10
b. valori de intrare și rezultate așteptate.....	10
6. Alte informații relevante pentru etapa de modelare și proiectare a proiectului.....	11

1. Diagrama UML a claselor

- clasele principale ale aplicației
- atributele și metodele
- relațiile dintre clase (moștenire, asociere, agregare, etc.)

Diagrama claselor de tip model:

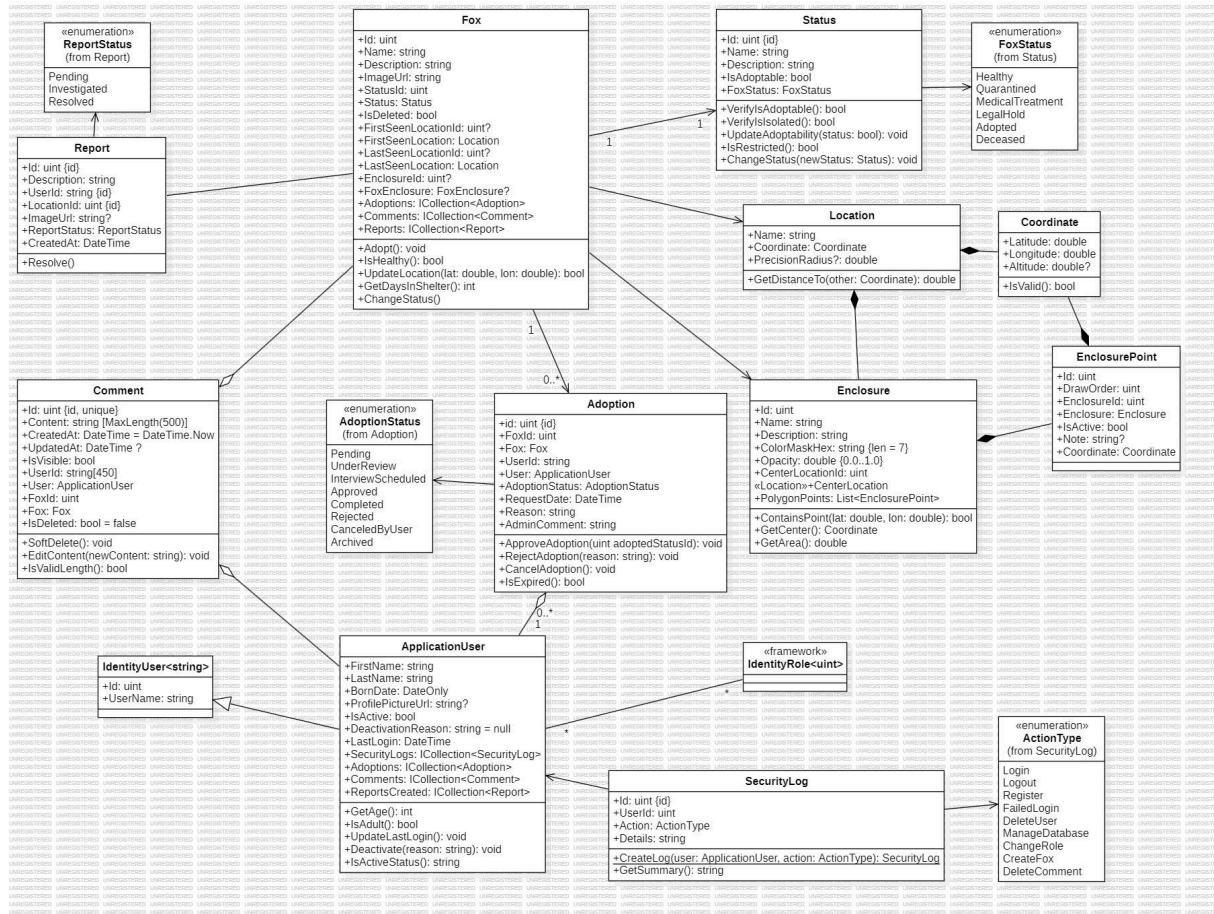


Fig 1. Diagrama Claselor Models UML

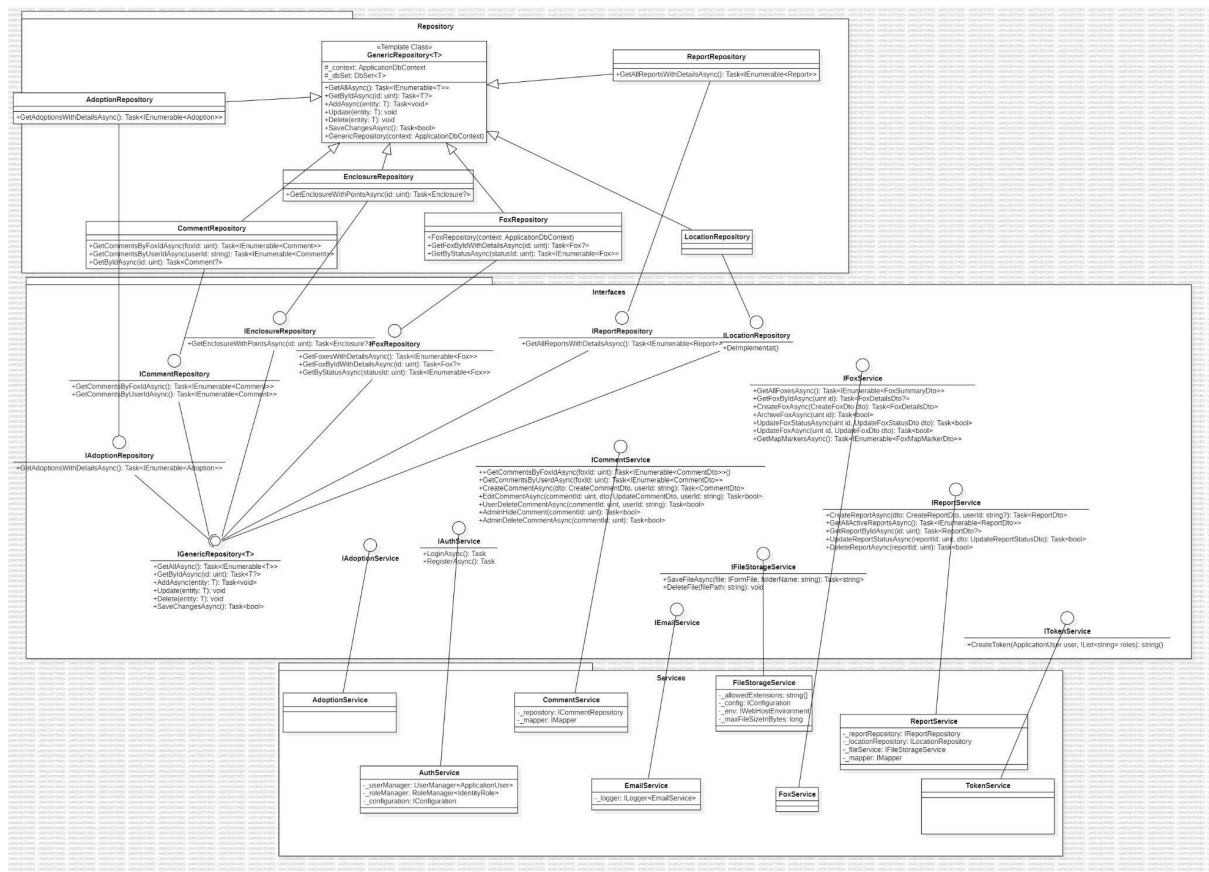


Fig 2. Diagrama Claselor Interfaces, Repositories, Services UML

2. Descrierea arhitecturii aplicației

a. structura pe module sau straturi (UI, logică, acces la date)

a.1. Stratul de Prezentare (UI)

Acest strat reprezintă interfața cu utilizatorul și este realizat folosind arhitectura MVC (Model-View-Controller).

Componente principale:

- ❖ Views (.cshtml) - Afășează informațiile către utilizator folosind HTML și CSS (Bootstrap).
- ❖ Controllers - Gestionează cererile utilizatorului și controlează navigarea între pagini.

Rol:

- ❖ afășarea datelor
- ❖ preluarea inputului de la utilizator
- ❖ comunicarea cu serviciile aplicației

a.2. Stratul API

Aplicația utilizează endpoint-uri REST API pentru comunicarea dintre frontend și server.

Rol:

- ❖ transmiterea și prelucrarea datelor;
- ❖ răspuns la cereri HTTP;
- ❖ conectarea interfeței cu logica aplicației.

a.3. Stratul de Logică

Acest strat conține logica principală a aplicației și regulile de business.

Rol:

- ❖ validarea datelor
- ❖ procesarea informațiilor
- ❖ implementarea regulilor aplicației
- ❖ gestionarea autentificării și securității

a.4. Stratul de Acces la Date (DAL)

Acest strat gestionează interacțiunea cu baza de date folosind Entity Framework Core.

Componente:

- ❖ Models – reprezintă entitățile din baza de date
- ❖ Repositories – gestionează operațiile de tip CRUD (add, delete, update, salvarea datelor, extragerea datelor din baza de date)

b. specificarea arhitecturii sau arhitecturilor folosite (MVVM, MVC, Client-Server, 3-tier, Clean Architecture etc.)

Arhitecturi și tehnologii folosite

- ❖ MVC (Model-View-Controller) - folosit pentru organizarea aplicației și separarea logicii de prezentare de logica aplicației.
- ❖ REST API - permite comunicarea dintre client și server prin cereri HTTP (HTTPS cu certificatul SSL).
- ❖ Entity Framework Core - folosit ca ORM pentru gestionarea bazei de date MySQL.
- ❖ Docker - aplicația poate fi rulată în containere Docker pentru o configurare și distribuire mai ușoară
- ❖ 3-tier - aplicația este împărțită în 3 straturi, stratul de prezentare, stratul de logică, și stratul de date.
- ❖ Principiile S & O - aplicația urmează principiile S (single responsibility class) și O (open-closed principle)

Aplicația este construită folosind o arhitectură de tip Client-Server. Serverul, dezvoltat în ASP.NET Core, gestionează logica aplicației și comunicarea cu baza de date, iar utilizatorii interacționează cu sistemul prin intermediul browserului web folosind protocoalele HTTP(S).

3. Designul interfeței grafice

a. schițe sau mockup-uri ale ferestrelor aplicației

schite:

Fig. 3.1, 3.2 Schiță Pagină Login + Pagină Rescued Foxes

din aplicație:

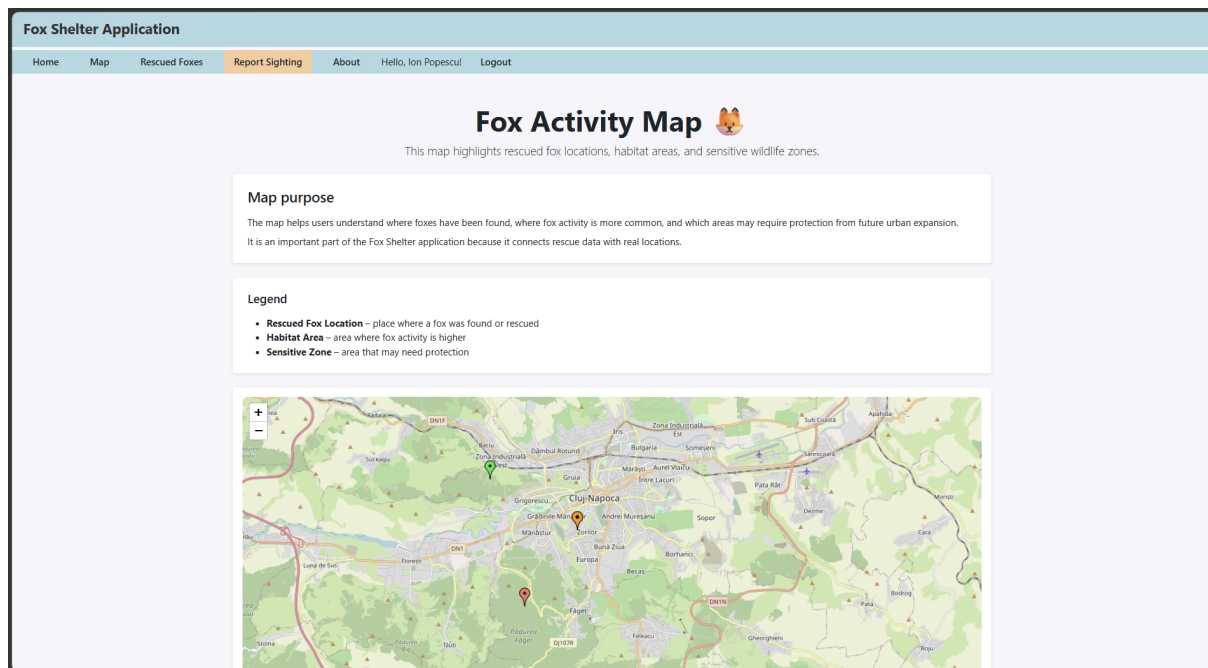


Fig 4. Pagină All Foxes Location

4. Descrierea claselor principale

a. rolul fiecărei clase + responsabilități

Modelele, care definesc structura datelor și regulile de bază ale obiectelor din sistem:

- ❖ **ApplicationUser**: Extinde sistemul standard de securitate (**IdentityUser**). Reține datele personale ale utilizatorilor (nume, prenume, data nașterii) și rolul acestora în sistem (Admin, Employee, User). Este „pilonul” central de care se leagă rapoartele, adopțiile și comentariile.
- ❖ **Fox**: Reprezintă entitatea centrală a adăpostului. Conține informații despre vulpea salvată (nume, descriere, imagine), și starea actuală a sănătății.
- ❖ **Adoption**: Gestionează procesul de adopție. Face legătura între un User (cel care vrea să adopte) și o vulpe. Reține data cererii, motivul adopției și statusul acesteia.
- ❖ **Report**: Salvează informațiile despre o vulpe văzută în libertate. Reține descrierea locului, coordonatele geografice (prin **Location**), o poză opțională și data la care a fost făcut raportul.
- ❖ **Status**: Definește stările în care se poate afla o vulpe. Această clasă dictează dacă o vulpe poate fi adoptată sau dacă trebuie izolată.
- ❖ **Enclosure (Țarc)**: Reprezintă o zonă geografică pe hartă delimitată pentru vulpi. Conține detalii despre nume, descriere și culoarea cu care va fi desenată zona pe harta interactivă.
- ❖ **Location & Coordinate**: Clase utilitare care gestionează punctele geografice (Latitudine/Longitudine) pentru a plasa vulpile pe hartă.
- ❖ **Comment**: Utilizatorii își pot exprima opinia cu privire la compania/comunitatea care folosește această aplicație.

- ❖ SecurityLog: O clasă care înregistrează automat acțiunile importante (login) pentru securitatea sistemului.

Controllerele (Stratul de Interfață) - se ocupă cu cererile de la utilizator.

- View Controllers (MVC):
 - ❖ HomeController: Gestionează paginile principale (Acasă, Despre, Hartă).
 - ❖ AccountController: Gestionează paginile de Login, Register și Profil.
 - ❖ ReportsController: Se ocupă de afișarea formularului prin care utilizatorii pot raporta vulpi.
- API Controllers:
 - ❖ FoxController & AdoptionController: Sunt folosite pentru a comunica rapid cu baza de date (fără a reîncărca toată pagina) pentru acțiuni precum filtrarea vulpilor sau procesarea adopțiilor.
 - ❖ + altele.

Serviciile (Stratul de Logică de Business)

- ❖ AuthService: Se ocupă de tot ce înseamnă securitate: verifică parolele, creează conturi și generează cheile de acces (JWT).
- ❖ FoxService & AdoptionService: Conțin regulile de business (ex: „O vulpe nu poate fi mutată într-un țărc plin” sau „Doar Admin-ul poate aproba o adopție”).
- ❖ FileStorageService: O clasă tehnică ce se ocupă cu salvarea pozelor încărcate de utilizatori în folderul de pe server.

Repositories

- ❖ GenericRepository<T>: O clasă generalizată care știe să facă operațiile de bază (Adăugare, Ștergere, Căutare) pentru orice model din listă, fără a rescrie codul de mai multe ori.
- ❖ FoxRepository & AdoptionRepository: Repoziitoare specifice care fac interogări mai complexe.

Alte Clase:

- ❖ ApplicationDbContext.
- ❖ DbInitializer: O clasă de „start” care, atunci când pornești aplicația prima dată, creează automat contul de Admin și câteva date de test.
- ❖ DTO-urile (Data Transfer Objects): Sunt niște „formulare” simplificate. În loc să trimitem toată informația despre un User, folosim un DTO care conține doar ce este strict necesar (ex: doar nume și email).

5. Planul de testare

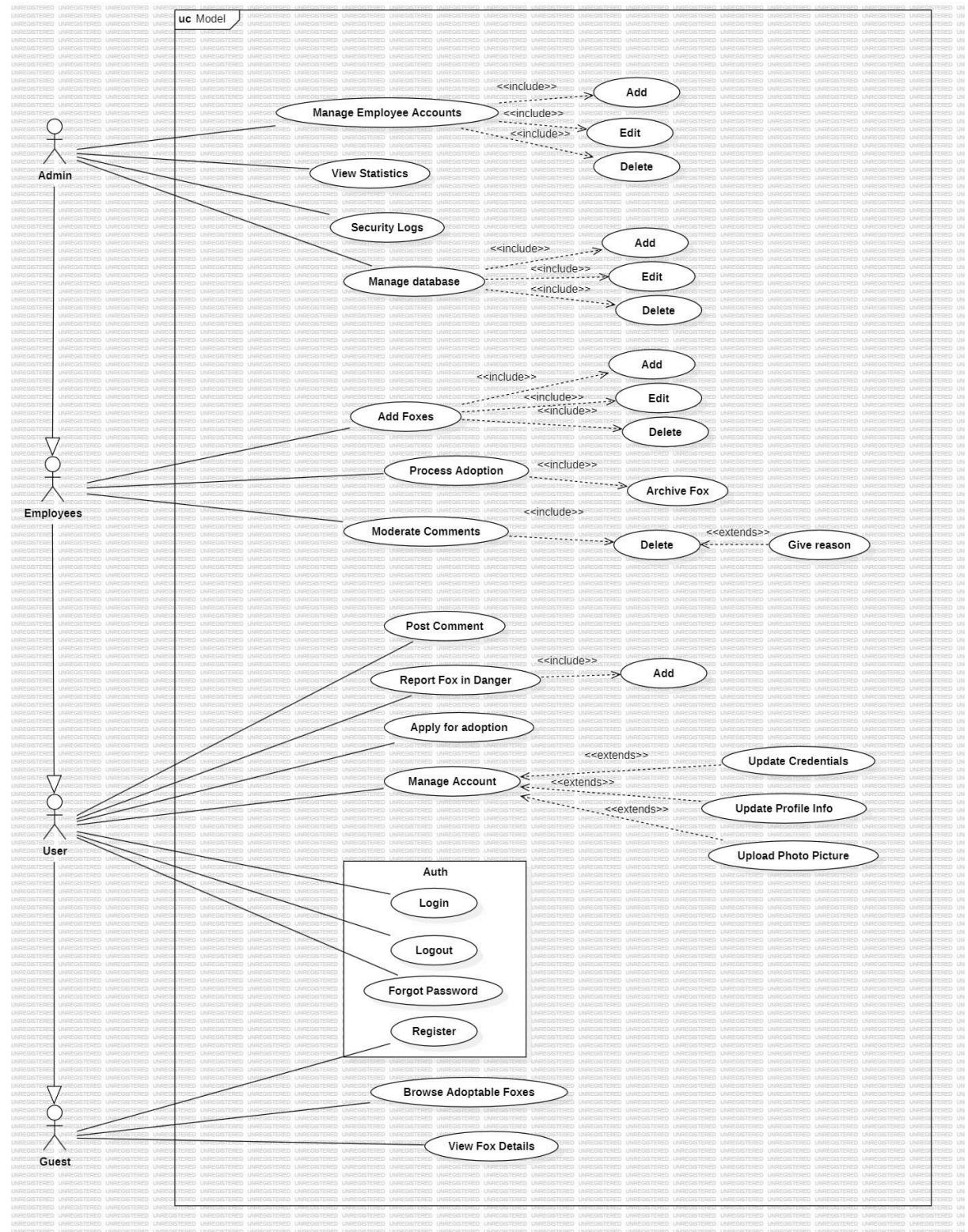


Fig. 5. Diagrama Use-Case

a. definirea cazurilor de testare

Pentru a asigura securitatea datelor, se verifică dacă acțiunile permise fiecărui tip de utilizator corespund cu Diagrama Use Case inclusă în documentație.

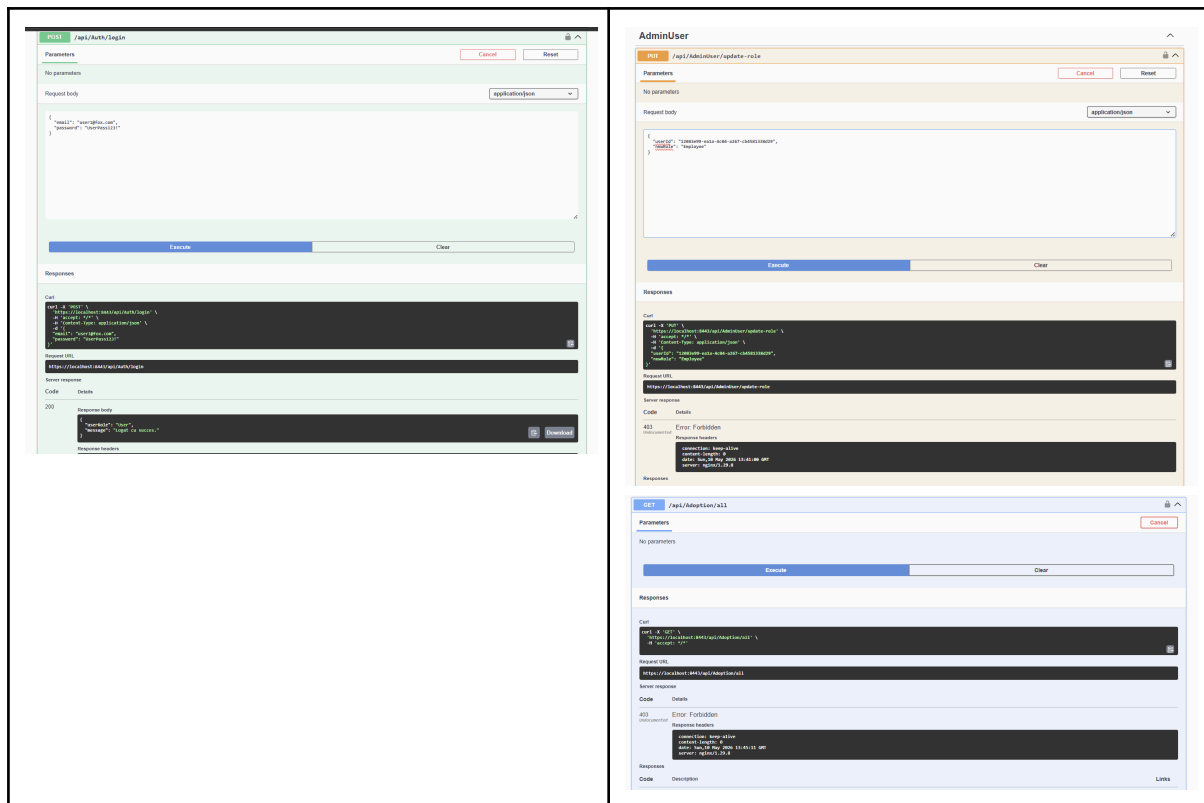
Scenarii de Testare a Permisunilor:

- ❖ Actor: Utilizator Neautentificat (Guest)
 - Test: Încercarea de a accesa pagina "Report Sighting" sau de a lăsa un comentariu.
- ❖ Actor: Utilizator Autentificat (User)
 - Test: Încercarea de a aproba o adopție prin API.
- ❖ Actor: Angajat (Employee)
 - Test: Verificarea accesului la funcțiile de actualizare a statusului unei vulpi și vizualizarea listei complete de adopții.
- ❖ Actor: Administrator (Admin)
 - Test: Verificarea accesului total (inclusiv ștergerea rapoartelor sau gestionarea conturilor altor utilizatori).

b. valori de intrare și rezultate așteptate

- ❖ Actor: Utilizator Neautentificat (Guest)
 - Test: încercarea de a accesa pagina "Report Sighting" sau de a lăsa un comentariu.
 - Rezultat Așteptat: redirectionare automată către pagina de Login.
- ❖ Actor: Utilizator Autentificat (User)
 - Test: încercarea de a aproba o adopție prin API.
 - Rezultat Așteptat: serverul trebuie să refuze cererea și să returneze un cod de eroare (403 Forbidden) sau să redirectioneze utilizatorul, deoarece aceste acțiuni sunt rezervate exclusiv adminului/angajaților.
- ❖ Actor: Angajat (Employee)
 - Test: verificarea accesului la funcțiile de actualizare a statusului unei vulpi și vizualizarea listei complete de adopții.
 - Rezultat Așteptat: acces permis (conform permisiunilor din diagrama Use Case), și salvarea în MySQL a datelor actualizate.
- ❖ Actor: Administrator (Admin)
 - Test: verificarea accesului total (inclusiv ștergerea rapoartelor sau gestionarea conturilor altor utilizatori).
 - Rezultat Așteptat: acces complet la toate modulele sistemului.

Exemplu: Un user nu poate modifica rolul unui alt user, sau să vizualizeze toate adopțiile.



Rezultatul returnat este o eroare 403 Forbidden, ceea ce este un rezultat așteptat.

6. Alte informații relevante pentru etapa de modelare și proiectare a proiectului

- a. **Securitatea Comunicațiilor: Protocolul HTTPS și Certificatele SSL:**

Un aspect fundamental al proiectării aplicației a fost asigurarea confidențialității datelor transmise între utilizator și server. Pentru a proteja datele sensibile (cum ar fi parolele și token-urile de sesiune), aplicația rulează exclusiv prin protocolul HTTPS.